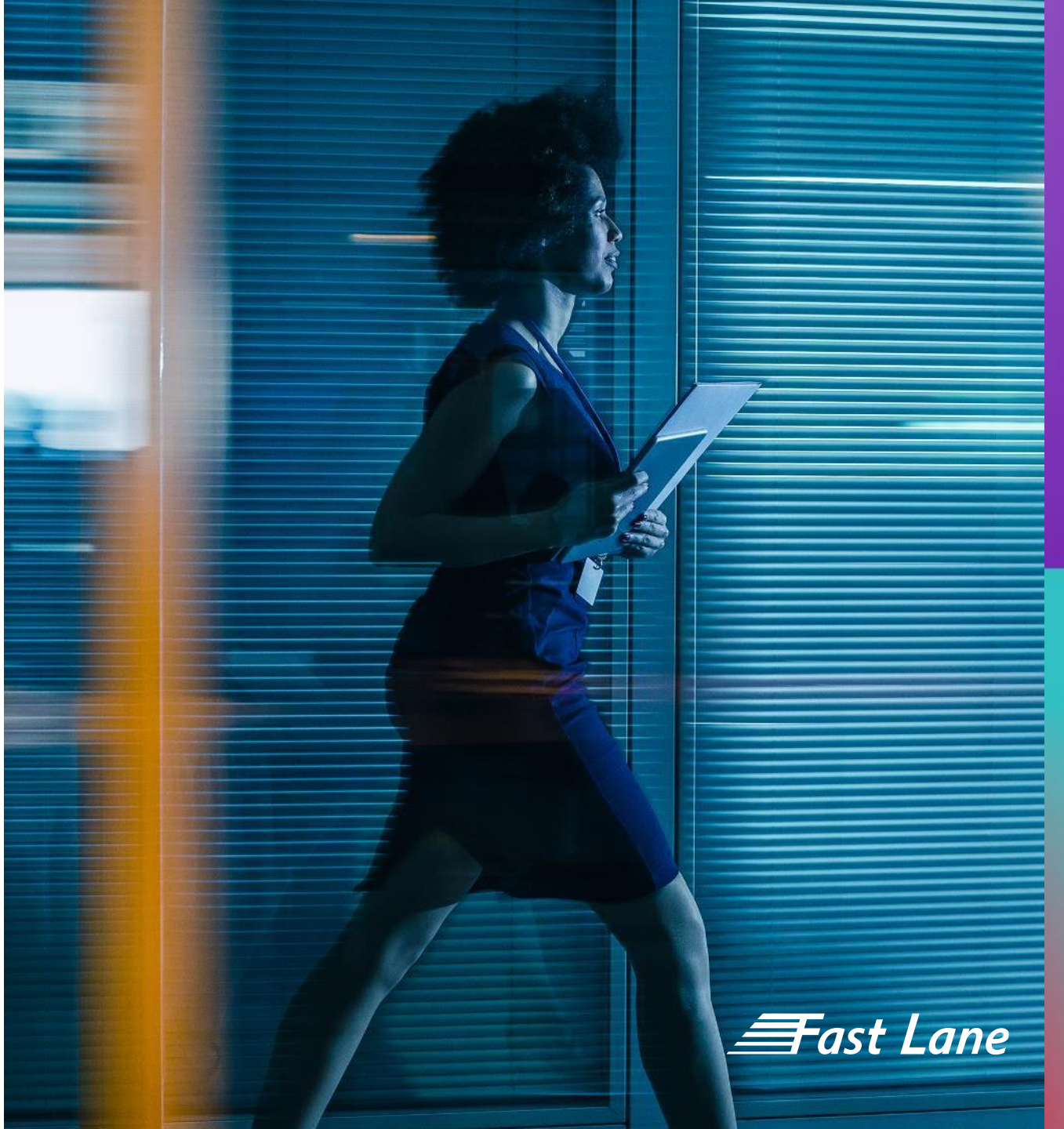
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 9

AI Agents in Event-Driven Systems

The exception-agent joins the Contoso Logistics event flow

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Microsoft.Extensions.AI: Foundry Model Client

IChatClient, Entra ID and middleware on every call

02

Agent Subscribers: React to Events

DeliveryDelayed in, RerouteRequested out, once per event

03

LLM Calls: Durable Activities

The model call as a retried, replay-safe activity

04

Rate Limits: Priority Queues & KEDA

Urgent work first, replicas capped by the quota

05

Vector Search: Change Feed Embeddings

Delivery notes found by meaning

06

Agent Choreography: Events vs. A2A

Published events or a direct call between agents

AI agents in event-driven systems

The exception-agent joins the Contoso Logistics event flow

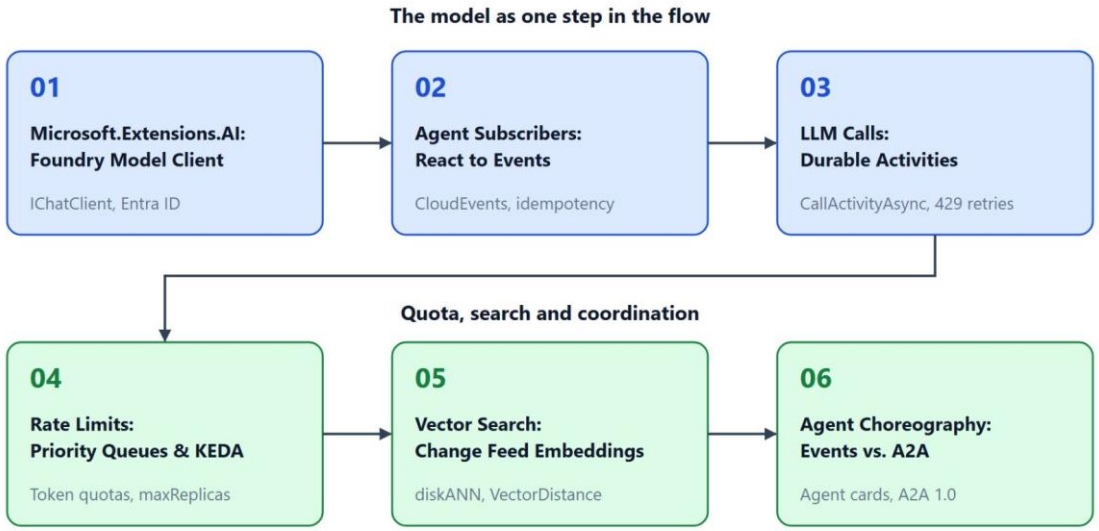
- **Calling the model**: IChatClient on a Foundry deployment, run from a subscriber and a durable activity
- **Staying in quota**: priority queues and a KEDA replica cap sized from tokens per minute
- **Search and coordination**: change feed embeddings, vector search, and events or A2A between agents



Overview

The path through the AI agents module

- Call a Foundry model through **IChatClient** with Entra ID and OpenTelemetry
- Subscribe the agent to DeliveryDelayed and keep it **idempotent**
- Run the model call as a **durable activity** that retries on 429
- Hold the agent inside its quota with **priority queues** and a KEDA cap
- Index delivery notes for **vector search**, then choose events or A2A



01

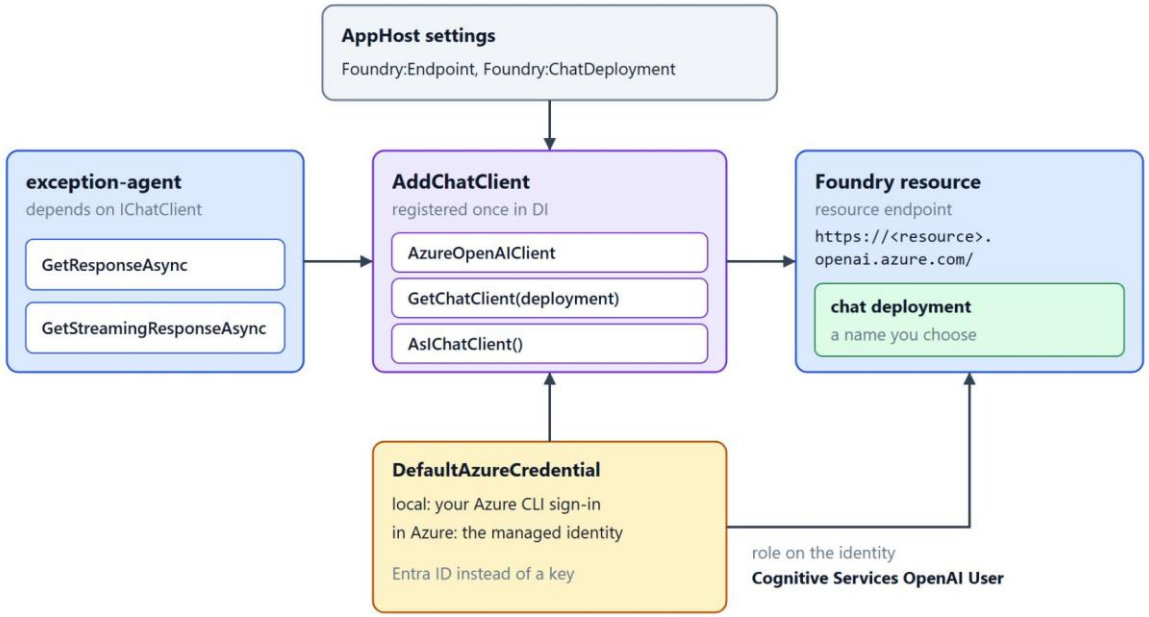
Microsoft.Extensions.AI: Foundry Model Client

IChatClient, Entra ID and middleware on every call

Connecting to a Foundry model deployment

IChatClient, DefaultAzureCredential and the resource endpoint

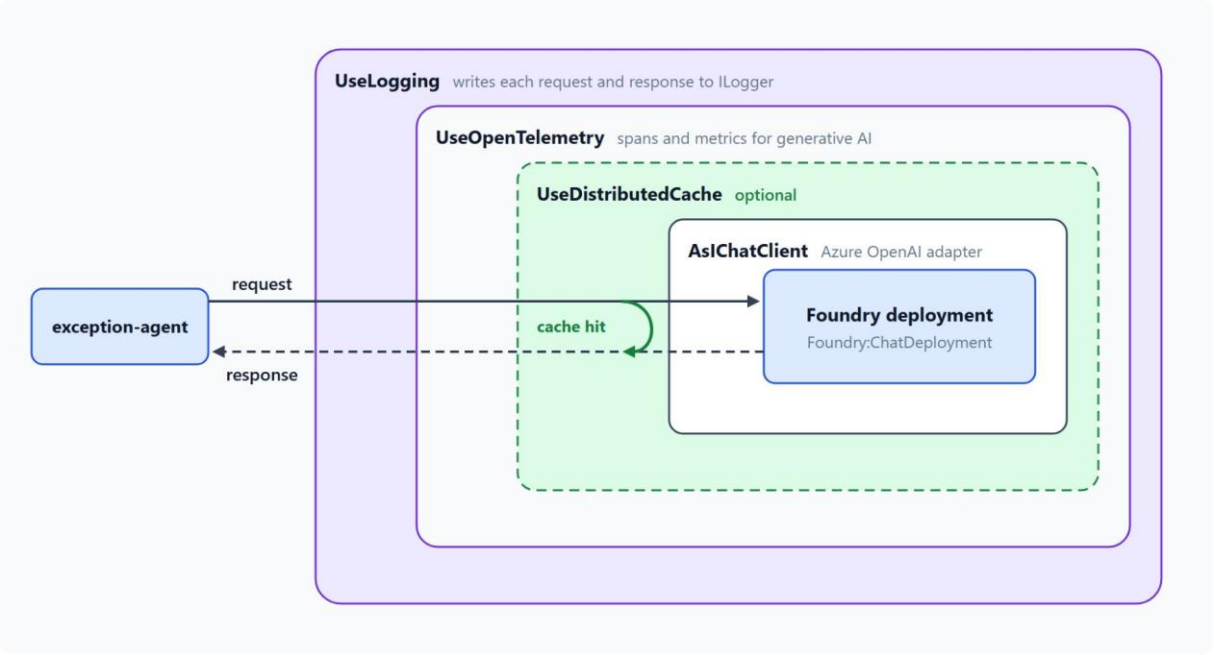
- **IChatClient** offers GetResponseAsync and GetStreamingResponseAsync, so the agent needs no vendor client
- `AzureOpenAIClient` takes the resource endpoint and `DefaultAzureCredential`, then `AsIChatClient`
- The identity holds **Cognitive Services OpenAI User**: Azure CLI sign-in locally, managed identity in Azure
- `AddChatClient` registers the pipeline once; the project endpoint is for Agent Framework clients
- `Aspire.Hosting.Foundry` (13.6.0-preview) adds `AddFoundry`; the lab passes `Foundry:Endpoint` and `Foundry:ChatDeployment`



Middleware on every model call

Logging and OpenTelemetry around the Foundry deployment

- `ChatClientBuilder` applies the Use methods in reverse order, so the first Use is outermost
- **UseLogging** writes each request and response through ILogger
- **UseOpenTelemetry** emits generative AI spans beside the HTTP and messaging spans
- The conventions are experimental: pin the package and check span names after each upgrade
- **UseDistributedCache** answers a repeated prompt without a model call; custom middleware derives from `DelegatingChatClient`



02

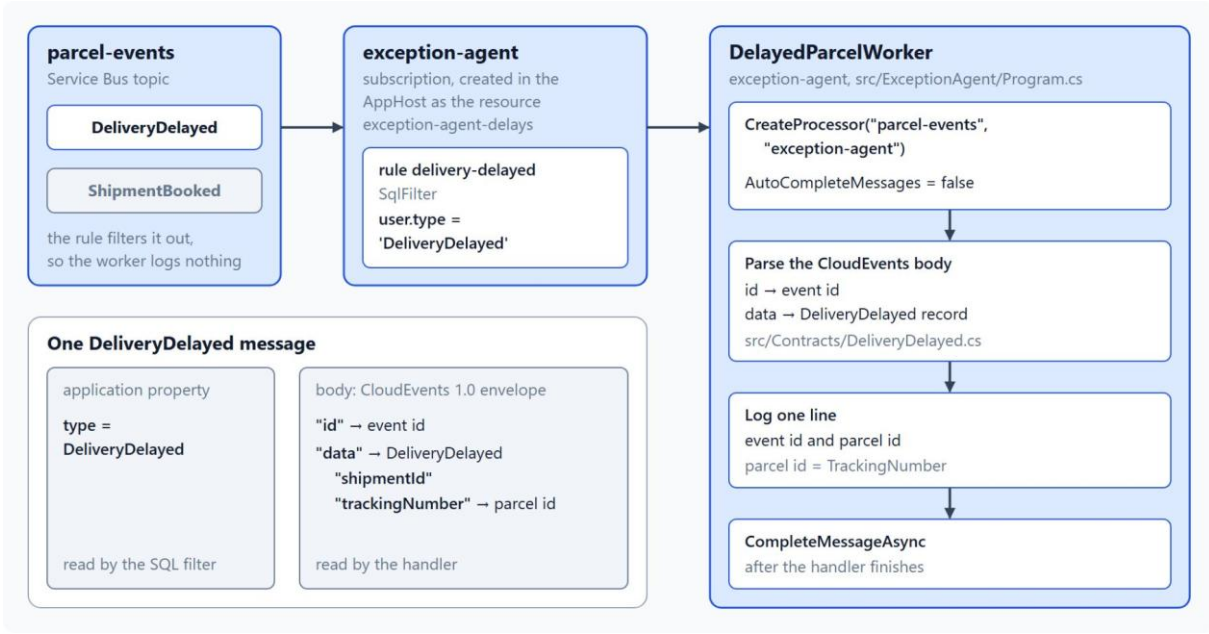
Agent Subscribers: React to Events

DeliveryDelayed in, RerouteRequested out, once per event

The agent as an event consumer

A filtered subscription and the CloudEvents envelope

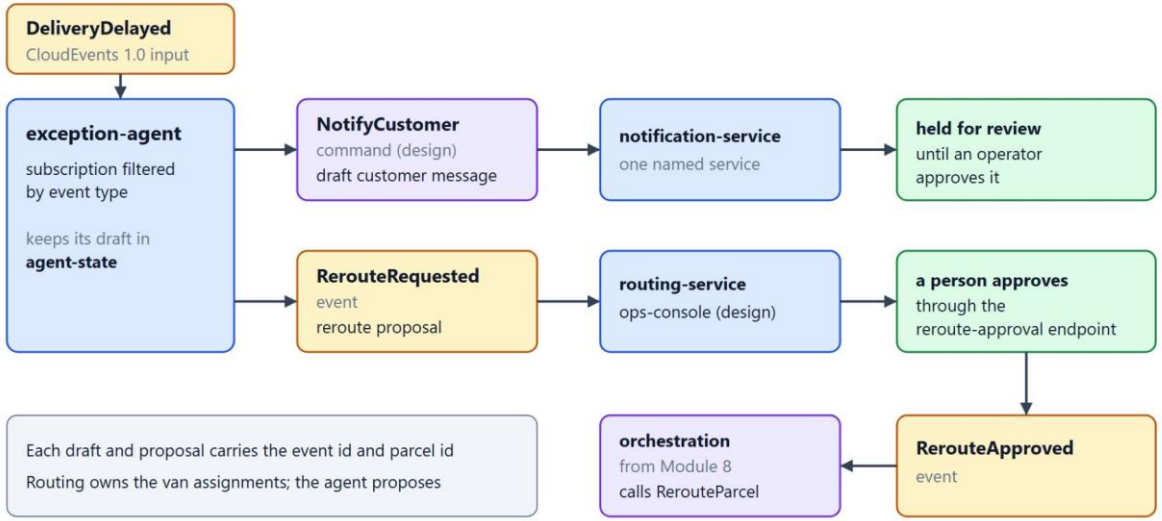
- A **topic subscription** with a SQL rule passes only DeliveryDelayed, the filter pattern from Module 4
- The body is a **CloudEvents 1.0** envelope: `id` is the event id, `data` holds shipmentId and trackingNumber
- The publisher owns the schema; the agent reads only the fields it needs
- A new optional field breaks nothing, a removed or renamed field does
- Customer details enter a prompt only when the draft needs them



Output as a command or an event

A draft for review, a reroute proposal for a person to approve

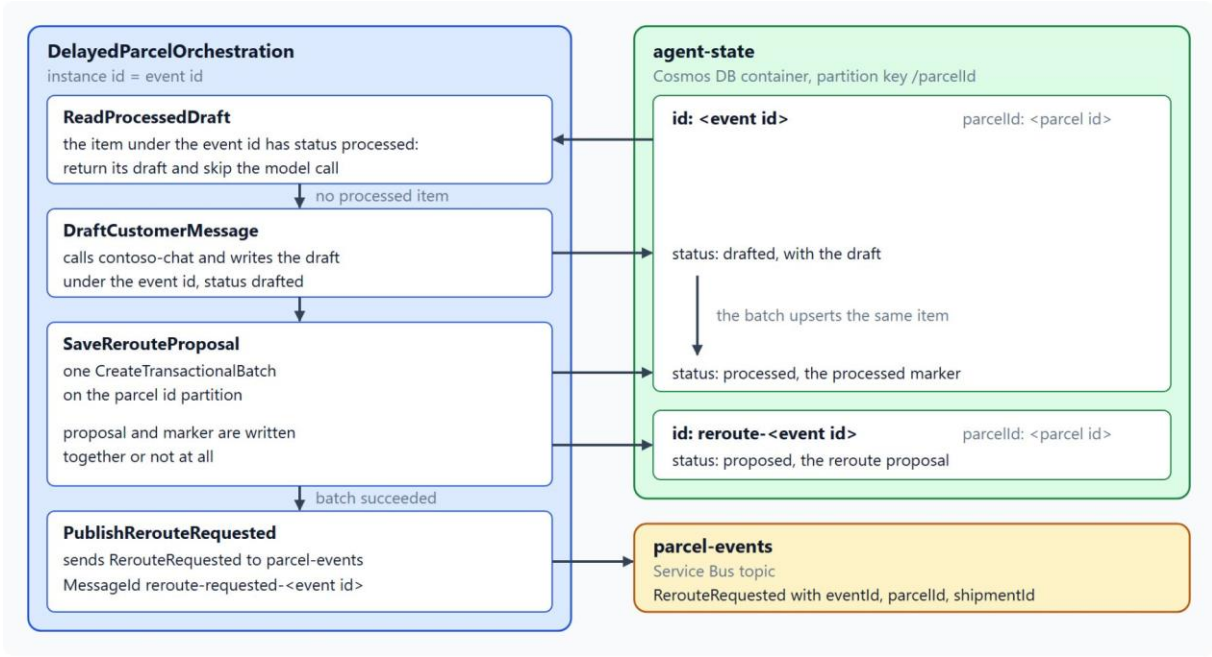
- A **command** asks one named service to act; an **event** states a fact any service can use
- The draft customer message is the `NotifyCustomer` command (design), held until an operator approves it
- The reroute proposal is the `RerouteRequested` event, the one message the lab's agent sends
- A person approves through the reroute-approval endpoint, which raises `RerouteApproved`
- The agent never calls RerouteParcel: the routing service owns the van assignments



Idempotency for agent consumers

Check the processed marker before the model call

- Brokers deliver **at least once**: duplicate detection does not stop a redelivery after a lock expires
- `ReadProcessedDraft` checks agent-state for the event id **before** the model call
- A retry reuses the stored draft, because a second model call words it differently
- `SaveRerouteProposal` writes the `reroute-` proposal and the processed marker in one transactional batch
- `RerouteRequested` goes to parcel-events only after the batch succeeds



03

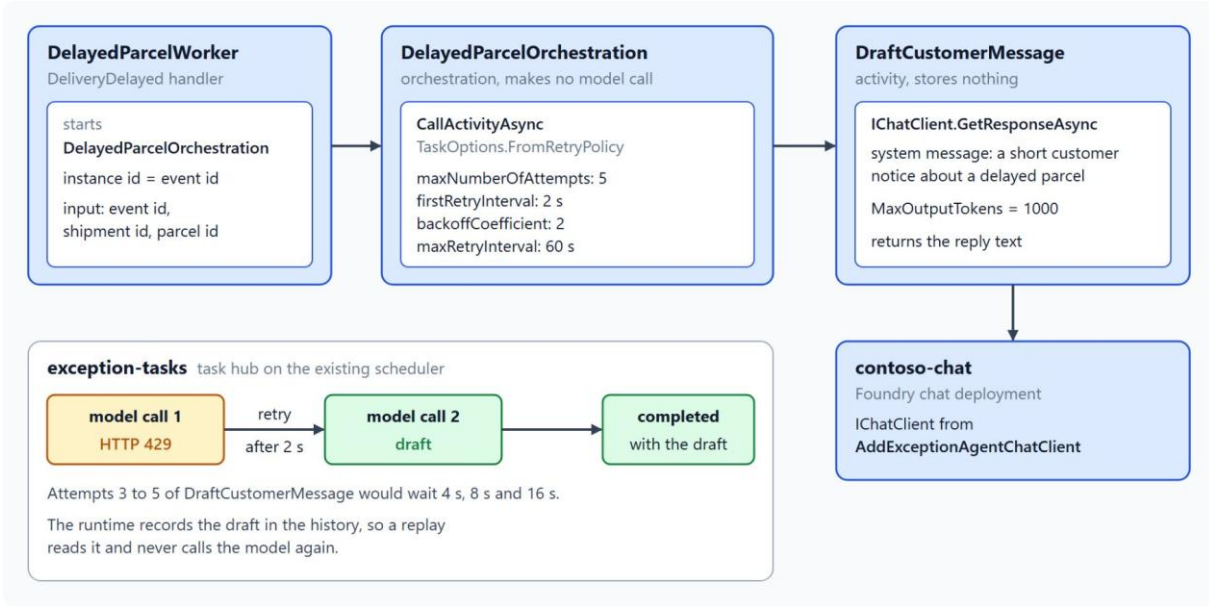
LLM Calls: Durable Activities

The model call as a retried, replay-safe activity

Why the model call belongs in an activity

Replay reads the recorded draft

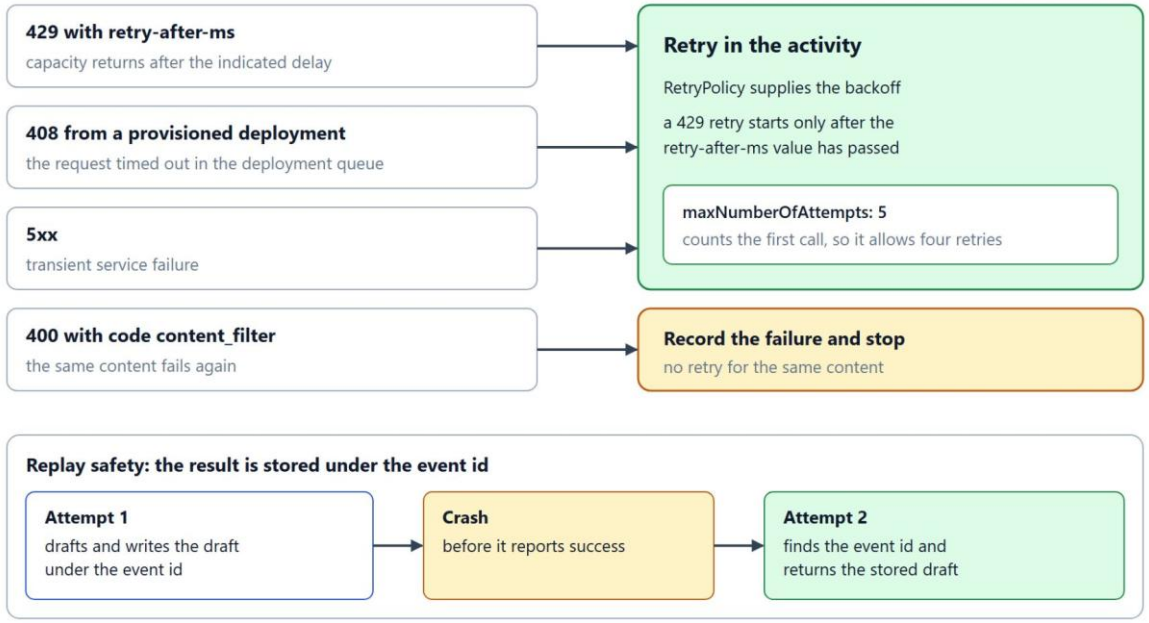
- An orchestrator **replays its history** each time it resumes, so every call must return the same result
- A model returns different text for the same prompt, which breaks the first replay
- An **activity** runs once per attempt, and the runtime records its output in the history
- `CallActivityAsync` with `TaskOptions.FromRetryPolicy`: 5 attempts, 2 s first interval, backoff 2, 60 s maximum
- A waiting orchestration has no model call in flight, so a parcel can wait a day for a person



Retries on throttling

Which responses retry, and how a retry reuses the stored result

- **429** carries retry-after-ms, and the next attempt waits at least that long
- **408** from a provisioned deployment and **5xx** retry as transient failures
- **400** with code content_filter fails again, so the activity records the failure and stops
- An attempt can crash after the model answered, and the runtime runs the activity again
- The activity stores its result under the event id and returns it on the next attempt



04

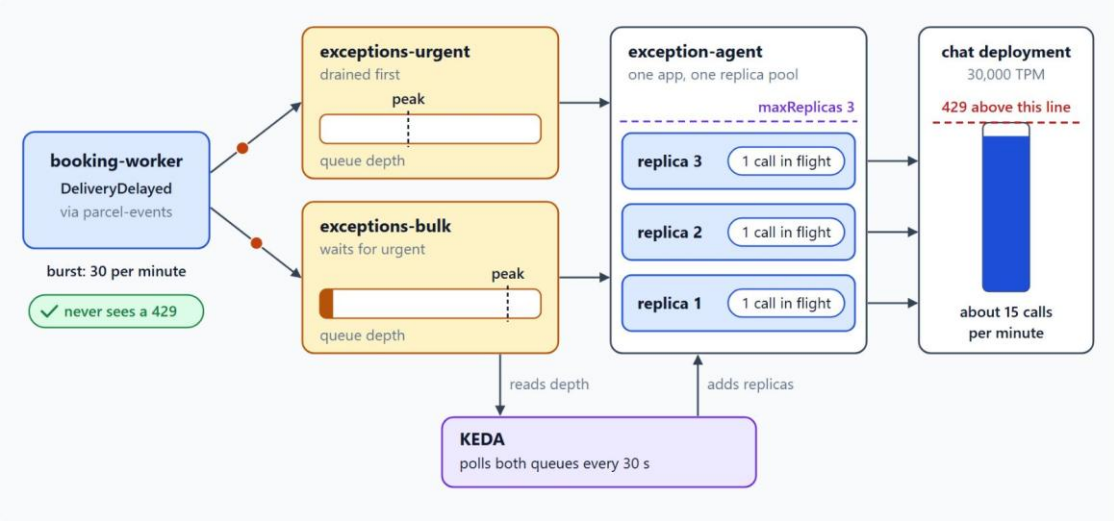
Rate Limits: Priority Queues & KEDA

Urgent work first, replicas capped by the quota

Model quotas and back pressure

The queues absorb a burst below the 429 line

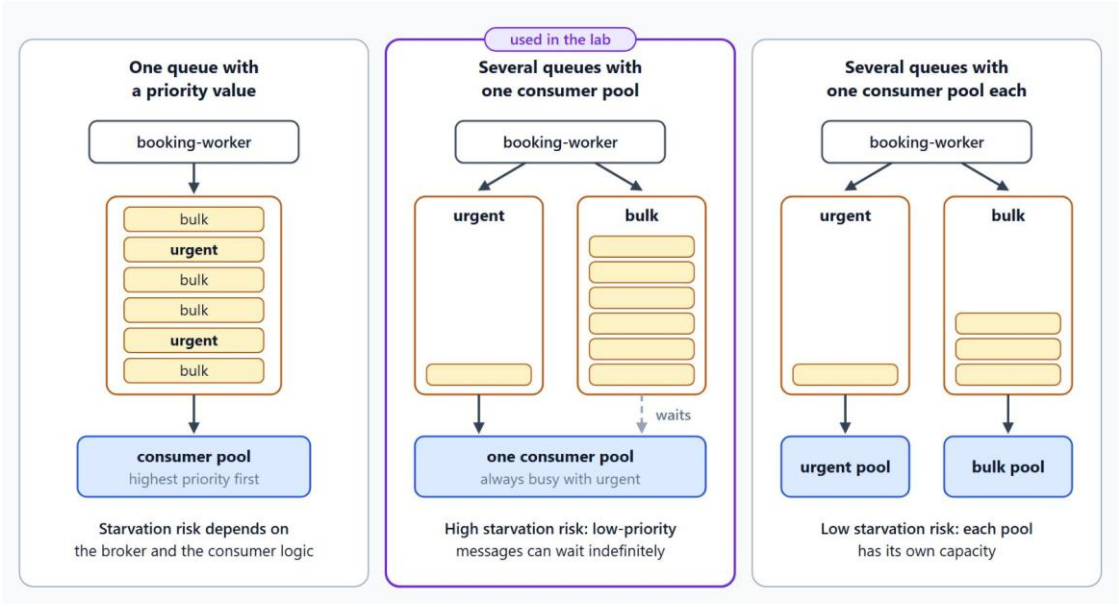
- Each deployment has a **TPM** allocation, the RPM limit follows from it, and either limit returns 429
- The token estimate counts the prompt plus **max_tokens**, so size max_tokens to a customer notice
- Failed requests count against the budget; back off with jitter and a fixed maximum
- The **queues absorb the burst**: booking-worker keeps publishing, the agent drains at the quota's rate
- Producers never see the 429; the backlog shows up as queue depth



Priority queues for urgent and bulk work

Three layouts and their starvation risk

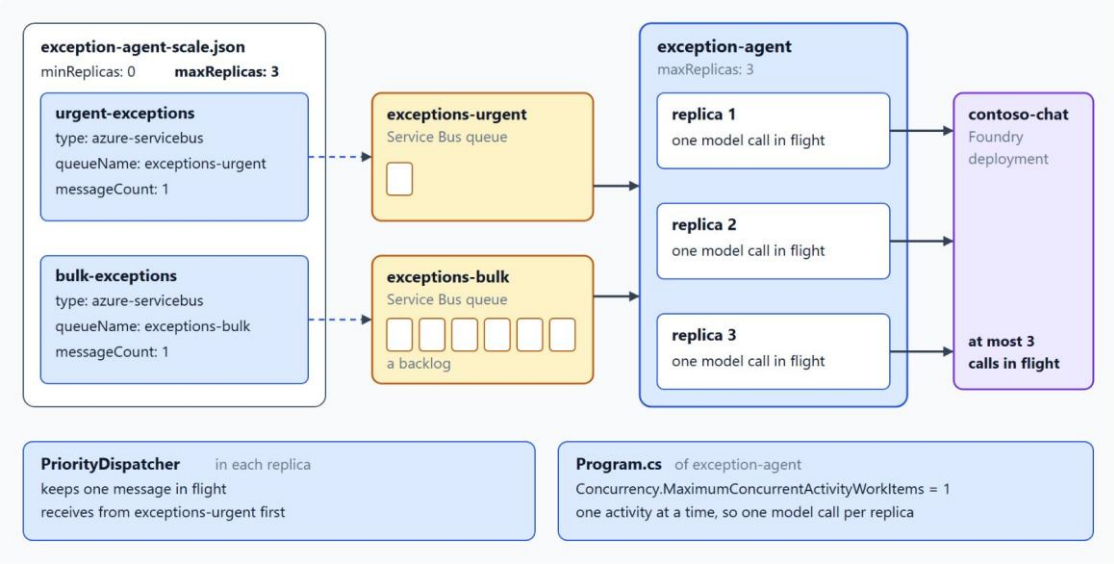
- **Urgent** exceptions need a draft within minutes; **bulk** sweeps can wait an hour
- One queue with a priority value: the risk depends on the broker and the consumer logic
- Several queues with one consumer pool: low-priority messages can wait indefinitely
- One pool per queue: each drains at its own pace, with low starvation risk
- The lab uses the middle layout, so a bulk backlog can still take every replica



Replica cap from the model quota

KEDA rules on both queues, at most three calls in flight

- An `azure-servicebus` rule scales on queue depth; **maxReplicas** is the fixed cap
- 30,000 TPM at about 2,000 tokens per call allows 15 calls per minute
- At 10 s per call, 2.5 calls run at once, so the cap is **three** replicas
- `MaximumConcurrentActivityWorkItems` of 1 keeps one model call in flight per replica
- API Management's **llm-token-limit** policy is the gateway option: 429 per minute, 403 when the quota runs out



05

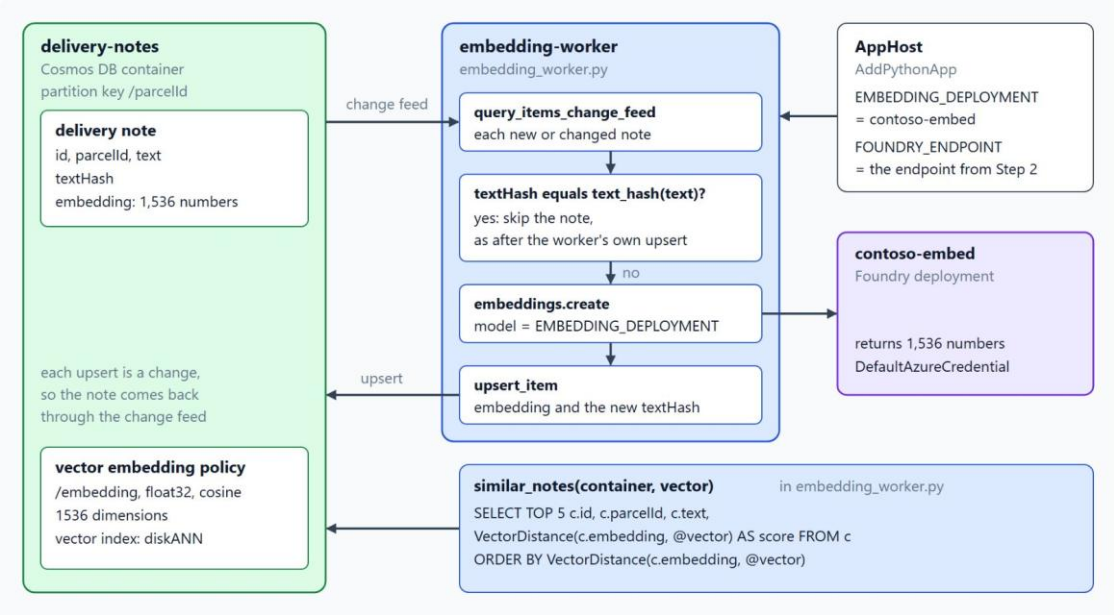
Vector Search: Change Feed Embeddings

Delivery notes found by meaning

Embeddings from the change feed

A text hash check ends the write-back loop

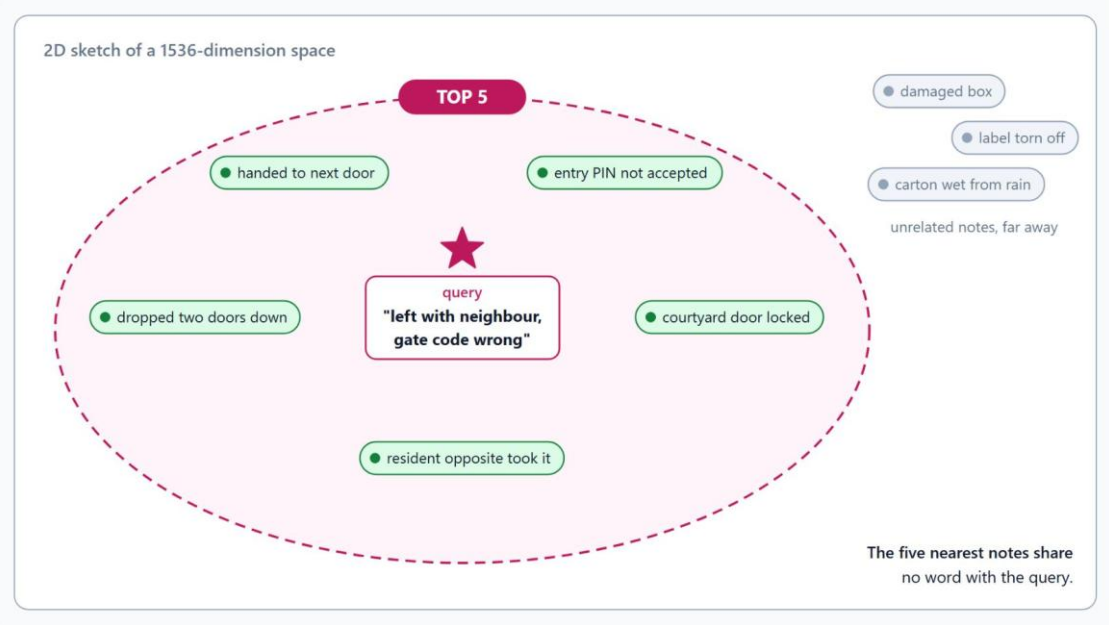
- Every insert or update of a delivery note appends a **change feed** entry
- The embedding-worker embeds each changed note; scanner and driver apps never call the model
- Writing the vector back is another change, so the worker skips notes whose **textHash** is unchanged
- A separate embeddings container removes the loop by construction
- Change feed consumers run at least once: a replay costs a model call and nothing else



Vector index and similarity query

DiskANN on /embedding, TOP 5 by VectorDistance

- `vectorEmbeddings`: /embedding, float32, 1536 dimensions, cosine distance
- A **diskANN** index on the same path; both policies are immutable after creation
- `flat` accepts at most 505 dimensions, so 1536 needs quantizedFlat for a brute-force scan
- `TOP 5 ... ORDER BY VectorDistance`; raise searchListSizeMultiplier when recall matters
- A model change means a new container and a full re-embedding run



06

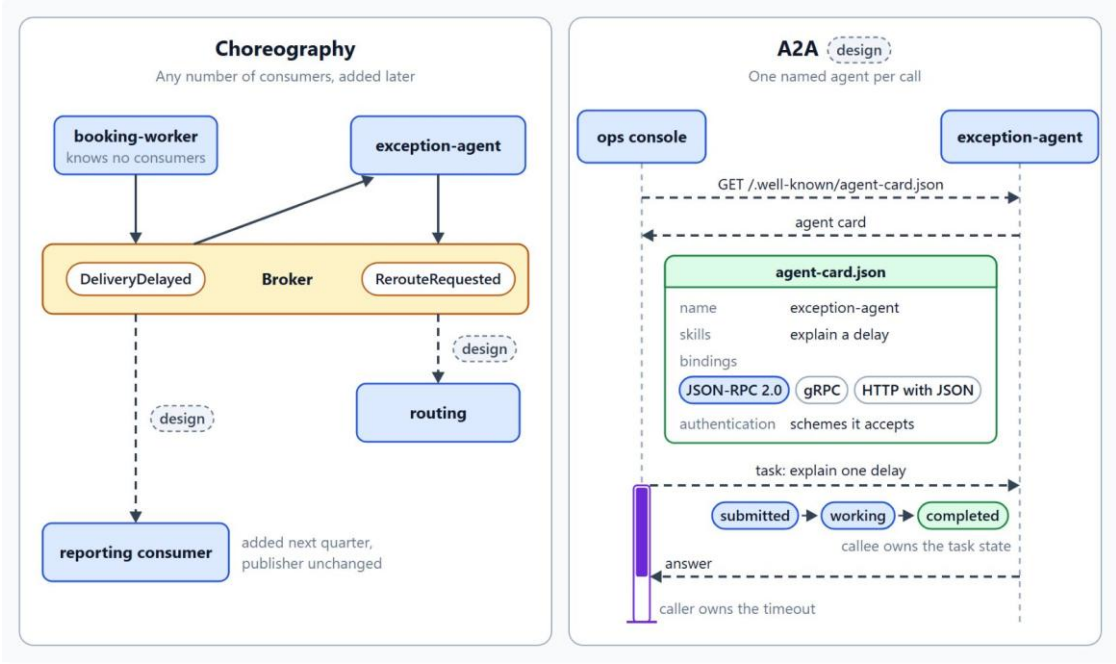
Agent Choreography: Events vs. A2A

Published events or a direct call between agents

Choreography and A2A

Published events or a direct call to one agent

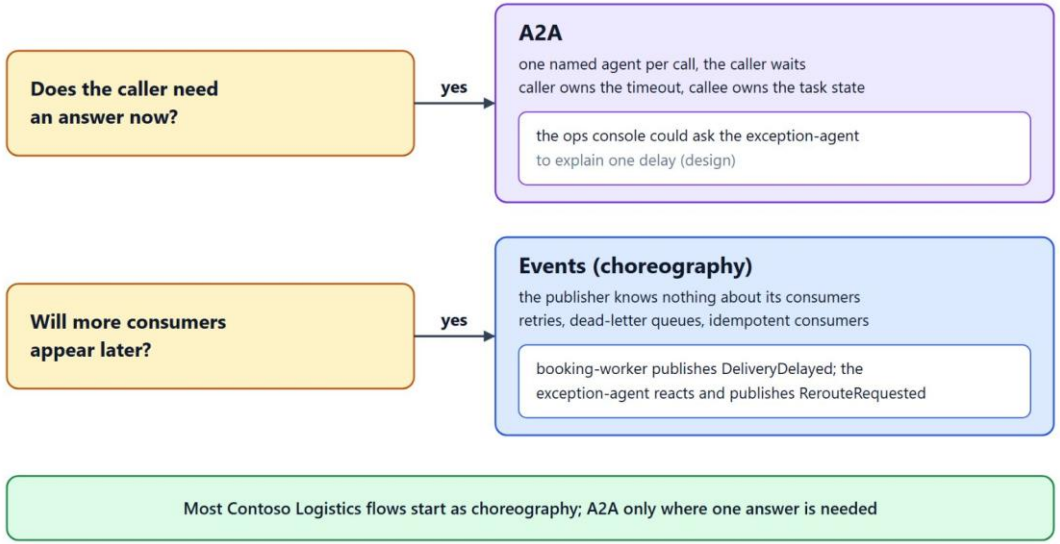
- **Choreography**: a producer publishes a fact, and every interested service subscribes
- Guard against event storms with event filters, consumer concurrency limits and explicit rules
- **A2A**: the callee publishes an agent card at `/.well-known/agent-card.json`
- A2A 1.0 defines JSON-RPC 2.0, gRPC and HTTP with JSON bindings, and permits custom ones
- The caller owns the timeout, the callee owns the task state



Choosing events or A2A

Two questions decide each interaction

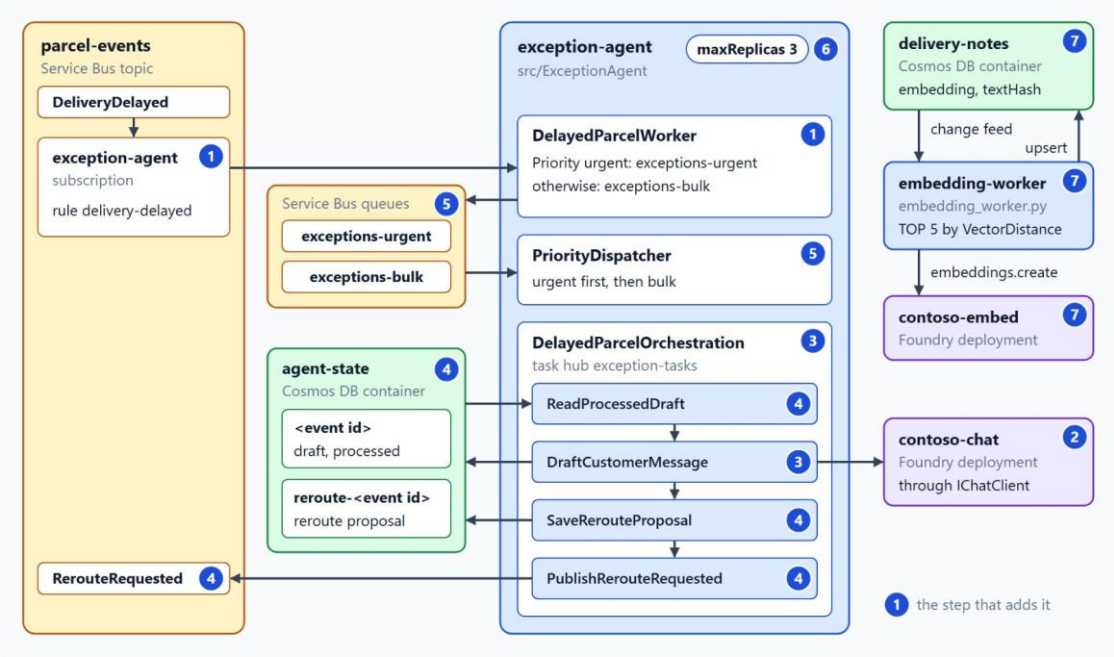
- Does the caller need an answer now? Then **A2A**
- Will more consumers appear later? Then **events**
- Most Contoso Logistics flows start as choreography
- A2A 1.0, announced on April 9, 2026, has more than 150 supporting organizations
- Since August 17, 2026, A2A is a hosted project of the Agentic AI Foundation



What the lab builds

An exception agent on DeliveryDelayed, in seven steps

- At the start no service reacts to DeliveryDelayed, and operators write the message and reroute by hand
- Steps 1 and 2: a filtered subscription and **IChatClient** on contoso-chat with Entra ID
- Steps 3 and 4: a durable draft activity with retries, and one batch per event in agent-state
- Steps 5 and 6: urgent work ahead of bulk, and KEDA capped at **three replicas**
- Step 7: a Python embedding-worker indexes delivery notes for VectorDistance search



Takeaways

The agent is a service like any other

- The exception-agent subscribes with the same contracts, retries and idempotency rules as any service
- Call the model through IChatClient with Entra ID, and only from a durable activity
- Check the processed marker before the model call, and reuse the stored draft
- Size replicas from the token quota and queue urgent work ahead of bulk
- Start agent coordination as choreography; use A2A when one answer is needed now

